King Penthouse ??? Master Supplemental R??sum??

King Penthouse — Master Supplemental Résumé Document

Source: everything in this repository (`kingpenthouse-cell/kingpenthouse1`) and the build conversation of Oct 17–23, 2026. Facts below were checked against the code. Anything marked **[YOU CONFIRM]** is something only you know (what you personally did vs. what a tool or I did). Do not put an item on a résumé unless you can explain it in an interview.

---

0. Read this first: how to describe your role honestly

The repository has two layers, and they came from different places:

1. **The public brand site (about 25 HTML pages, ~65 images and videos, in the repo since Aug 31, 2026).** The files contain markers showing they were generated or assembled with an AI design tool: a custom component runtime (`support.js`, 1,911 lines, `<x-dc>` tags, `class Component extends DCLogic`) and a starter scaffold (`image-slot.js`, 1,225 lines, labelled "omelette starter scaffold"). It is not hand-written React or Vue. Your contribution is likely product direction, art direction, content, event data entry, and deployment. **[YOU CONFIRM what you wrote or edited yourself.]**
2. **The `/kp` flyer landing page and its notification backend (built Oct 17, 2026).** The page design came from your design package. The serverless function, validation, security, email and WhatsApp integration, tests and config were written by Claude Code at your direction.

Safe, accurate framing: **"Founder/operator and technical product owner who specified, directed and shipped a production web funnel using AI-assisted development."** That is a real and increasingly valued skill. Claiming "I hand-coded a serverless backend" is not accurate unless you did.

---

1. What the project is

King Penthouse is a real, operating, invite-style nightlife brand: private after-hours themed parties for adults 19+ in Mississauga, Ontario, near Toronto Pearson Airport. The venue address is deliberately never public; guests must be verified first.

The website is the brand's front door and its lead-generation system. The `/kp` page is a **QR-code landing page for physical flyers** handed out at events and venues (Toronto Sex Show, Sugar Daddy's). Its job is to convert a scan into a verification request, and to tell the owner which flyer channel produced it.

Business context worth stating on a résumé: this is a live product with real customers, a real privacy constraint (never expose address or owner phone), real marketing spend (print flyers), and a measurable question (which channel works?).

---

2. The site: pages and features (verified in the repo)

Page	What it does
`index.html`	Cinematic hero landing: animated logo splash (once per session), full-screen hero image, scrolling ticker of upcoming events built from data, entry buttons
`Events.html` / `events.html`	Event listing generated from a data file; past events drop off automatically
Per-event pages (e.g. `Oct17-Jungle-Fever.html`, `Oct3-King-Of-Spades.html`, about 14 total)	One detailed page per party with poster, time, prices, details, house rules
`halloween.html` (+ `Oct31-Halloween.html` redirect)	Flagship seasonal landing page with its own palette, video, imagery
`Location.html`	Venue photo gallery with scroll-triggered reveal animations (`IntersectionObserver`), address kept private
`Reviews.html`	Social proof page with review board imagery
`Contact.html`	Contact page with a copy-to-clipboard email button and links to verification and FetLife
`Verification.html`	Multi-field guest verification form posting to Netlify Forms (`data-netlify`), using `fetch` for in-page submit with abort/timeout handling, file upload support (`multipart/form-data`)
`kp.html` (new)	QR flyer landing page: nav, hero ("So you found our flyer?"), lounge photo, request form, success state

Single source of truth for events (`events.js`, 353 lines): one JavaScript data file drives the ticker on the home page, the Events page and the Halloween content. Each event has slug, category, name, tagline, date, time, prices, poster, details sections, and a `status` of `active` or `hidden`. Past events are filtered out automatically (with a 6-hour grace window for after-midnight events). To add an event you copy a block and create a page. This is a lightweight content-management pattern with no database.

Shared standard content: a reusable `KP_STD_DETAILS` block (what to expect, single men policy, consent, privacy, the space, verification, house rules) is injected into every event page so policy wording stays consistent.

---

3. The `/kp` feature set (the part with real engineering depth)

3.1 Front end

- Mobile-first single-column layout (max width 480px), sticky navigation, large 52–56px tap targets, honoring phone use from QR scans.

- Form fields: name/alias; contact method toggle (Email or WhatsApp); contact field whose placeholder, input type and keyboard mode change with the choice (`type=tel` vs `email`, `inputmode`, `autocomplete`); "attending as" selector; "how did you find us" selector with a free-text "Other"; required 19+ confirmation.
- Client-side validation with specific, friendly messages; submit button disabled and relabelled "SENDING…" during requests; generic retry message on failure; success screen "REQUEST RECEIVED" replaces the form.
- Hidden honeypot field to catch bots.
- Reads `?source=` from the QR URL, validates it, and sends it with the form.
- Page is `noindex` so it is only reachable by flyer, not search.

3.2 Flyer attribution tracking (the best product idea in the project)

Three different QR codes route to the same page with different `source` values: - `kingpenthouse.com/kp?source=sexshow` - `kingpenthouse.com/kp?source=sugardaddys` - `kingpenthouse.com/kp?source=flyer`

Every submission carries the source into both admin notifications, plus the guest's self-reported "how did you find us". This gives **two independent measurements** of channel performance (what the QR said vs. what the guest said) and lets you answer "did the flyers work, and which venue should I print for again?" with numbers.

3.3 Serverless backend (`netlify/functions/kp-rsvp.mjs` and `lib/`)

Node.js 18+ ES-module function on Netlify Functions (v2 Request/Response API), no third-party dependencies.

- **Validation:** allow-lists for every dropdown; email regex; phone length (10–15 digits); required 19+ confirmation checked on the server, not just the browser.
- **Sanitization:** Unicode normalization (NFKC), removal of control characters, zero-width and bidirectional-override characters, angle brackets stripped, whitespace collapsed, length caps. Prevents HTML/script injection and email header injection (newline injection in subjects).
- **Bot defense:** honeypot returns a fake success so bots cannot tell they were caught.
- **Abuse limits:** per-IP rate limiting (5 per 10 minutes), request body cap (8KB), same-origin check on the `Origin` header, POST-only.
- **Two notification channels sent in parallel with `Promise.allSettled`:**
  - Email through the **Resend** REST API (HTML + plain-text versions, HTML-escaped, `reply_to` set to the guest when they chose email, timestamp in America/Toronto).
  - WhatsApp admin alert through the **WhatsApp Business Cloud API** (Meta Graph API): approved-template messages with 7 ordered body variables, configurable template name/language/variable order from environment variables, automatic fallback to free-form text when no template is configured, and sanitizing to meet Meta's template rules (no newlines, tabs or long space runs).
- **Failure isolation:** one channel failing never blocks the other. Email OK + WhatsApp failed (or the reverse) still accepts the RSVP and logs the failure server-side. Both failing returns a generic error so the lead is not silently lost, and logs a CRITICAL line.
- **Zero information leakage:** the browser only ever receives `{ok:true}` or `{ok:false}`. No provider names, error text, phone numbers or keys.
- **Secrets management:** all credentials come from Netlify environment variables (8 of them). Nothing private exists in the repository, HTML or browser JavaScript. An automated test scans `kp.html` for phone numbers, `wa.me`, `tel:`, `mailto:` and variable names.

3.4 Testing

26 automated tests using Node's built-in test runner (`npm test`, no framework needed): - valid submissions (email and WhatsApp contact, "Other" with text) - 11 invalid-input rejection cases, with a check that nothing is sent - honeypot, malformed JSON, oversized body, wrong HTTP method, foreign origin - rate limit (6th request returns 429) - injection and header-injection sanitization - the three failure modes (email down, WhatsApp down, both down) including a check that error text never reaches the visitor - exact request shape sent to Resend and to Meta, with the provider calls mocked - a privacy test over the shipped HTML

Honest limit: provider calls were mocked. The integration was **not yet verified against live Resend/Meta accounts**; that verification is Step 5 of the Cowork setup prompt.

3.5 Deployment configuration

`netlify.toml` (publish directory, functions directory, esbuild bundler, `/kp` route rewrite preserving query strings, security headers `X-Content-Type-Options`, `Referrer-Policy`, `X-Robots-Tag`, no-store caching on the function), `.env.example`, `.gitignore`, `package.json` with a test script and Node engine requirement.

---

4. Skills demonstrated (grouped, with evidence)

Product and business

- **Product ownership of a live product:** defined audience (flyer scanners), goal (verification request), constraints (privacy, 19+), and success metric (leads per channel).
- **Funnel design and attribution:** per-channel QR codes, dual attribution (URL source + self-reported), designing to answer a marketing question.
- **Privacy-by-design thinking:** you specified, up front, that the admin phone, admin email and venue address must never reach the browser, HTML, metadata, logs, source maps or API responses. That is a security requirement most clients forget.
- **Operations and trust design:** a verification step, consent-first house rules, 19+ gating, discreet communications. Community safety policy is part of the product.
- **Brand building and positioning:** upscale/exclusive/private tone, restrained gold-and-black palette, seasonal sub-brand (Halloween purple/orange/gold).
- **Event operations and content management:** run a recurring event calendar with pricing tiers, capacity messaging, and event-specific rules.
- **Marketing channel management:** physical flyers, venue partnerships, trade-show presence, FetLife/social presence.

Design / UX

- Mobile-first, QR-scan-first design; tap-target sizing; keyboard types per input; low-friction short form (about 6 inputs) for cold traffic.
- Progressive disclosure, clear single call to action, success state that confirms without exposing internals.
- Luxury visual language: serif/sans pairing (Playfair Display, Cormorant Garamond, Jost, Outfit), gold gradients, thin borders, generous spacing, subtle animation (logo reveal, scroll reveals, ticker).

- Consistent design system across 20+ pages (shared palette, type, nav).

Engineering (what the code demonstrates; specify your own part)

- Serverless architecture (Netlify Functions), REST API integration (Resend, Meta Graph API), JSON over HTTPS.
- Secure input handling: allow-listing, canonicalization, injection and header-injection defense, output escaping, honeypots, rate limiting, origin checks, body-size limits.
- Resilient design: independent channels, `Promise.allSettled`, timeouts via `AbortSignal.timeout`, graceful degradation, observability through server-side logs only.
- Secrets management with environment variables; separation of public and private config.
- Testing: dependency injection for testable handlers, mocked network, 26 tests, a regression test for privacy.
- Static-site architecture with a data-driven content layer (`events.js`), URL rewrites/redirects, Netlify Forms.
- Front-end JavaScript: DOM events, `fetch`, `URLSearchParams`, `IntersectionObserver`, `sessionStorage`, clipboard, `localStorage`-safe try/catch.
- Git and GitHub: feature-branch workflow, commits and pushes (`claude/king-penthouse-landing-hemgbq`).

AI-assisted development workflow (a real, résumé-worthy skill)

- Writing a precise technical spec (your original brief): routes, form fields, environment variable names, security requirements, failure semantics, privacy constraints, test expectations. The quality of that spec is the reason the output was usable.
- Iterating the spec mid-flight (you added WhatsApp, the Sugar Daddy's / Sex Show tracking, and the `?source=` parameter, which are all good product decisions).
- Working across multiple AI tools: a design tool produced the page design package, a coding agent built and tested the backend, and a browser agent (Claude Cowork) is being used for account and deployment setup. Handing documents between tools (design handoff markdown, setup prompt) is itself a workflow skill.
- Reviewing and integrating output: reconciling the designer's form contract with the backend contract (adapting the function to the final design rather than restyling it).
- Delegating operational work with explicit guardrails (the Cowork prompt: never print secrets, ask before spending money, ask before merging).

---

5. What is unique or impressive

1. **It is a real business with a real constraint.** Most portfolio sites are demos. This one has live traffic, a marketing channel, and genuine privacy stakes.
2. **Privacy engineered as a requirement, with an automated test enforcing it.** Not just "I didn't put the number on the page".
3. **Dual-channel notification with true failure isolation.** Email and WhatsApp are independent; either can fail and you still get the lead. That is production thinking.
4. **Marketing measurement baked into the product.** Per-flyer QR attribution plus self-reported source, so print spend becomes a data-driven decision.
5. **Zero-dependency backend.** No `node_modules` to maintain or exploit; uses only platform `fetch` and `Intl`.
6. **Bot defense that does not hurt real users:** silent honeypot, no CAPTCHA friction on a mobile QR flow.

7. **Non-engineer shipping production-grade infrastructure** through clear specification, which is exactly the skill modern teams look for.
8. **Data-driven content layer** that lets a non-developer add a party by copying one block.

---

6. Things you can build with this skill set

- Lead-capture and RSVP funnels for events, venues, restaurants, clubs, promoters, festivals.
- QR-code marketing campaigns with per-channel attribution and reporting.
- Booking, waitlist, membership-application and verification flows.
- Private/invite-only community sites and applicant screening forms.
- Event-ticketing front ends and event calendars driven by a data file or CMS.
- Notification systems: email + SMS/WhatsApp/Slack/Discord alerts for form submissions, orders, or alerts.
- Serverless APIs and webhooks on Netlify/Vercel/Cloudflare; integrations with Stripe, Resend, Twilio, Airtable, Google Sheets.
- Brand and campaign landing pages for local businesses.
- Admin dashboards (a logical next step: store submissions in a database and chart leads by channel).

---

7. Roles this is relevant to

Product owner/manager (growth or marketing tech), growth/marketing operations, technical founder or operator, junior full-stack or front-end developer (with honest framing of AI assistance), no-code/low-code or automation specialist, technical project coordinator, e-commerce or web operations, community/event operations manager, digital marketing specialist (attribution, funnels), customer onboarding and verification/trust-and-safety roles (verification workflow design), solutions/implementation consultant.

---

8. Ready-to-adapt résumé bullets (edit to match what you really did)

- Founded and operate **King Penthouse**, a 19+ private event brand in the Greater Toronto Area; own brand, marketing, event operations, guest verification and the website. **[add your real numbers: events per month, average attendance, years running]**
- Specified and shipped a QR-code flyer funnel with per-channel attribution (trade show, venue, general flyer), enabling data-driven decisions on print marketing spend.
- Directed development of a serverless RSVP backend (Netlify Functions, Node.js) that validates and sanitizes submissions server-side and notifies the admin by email (Resend) and WhatsApp Business Cloud API in parallel, with independent failure handling.
- Defined and enforced a privacy requirement that the venue address and admin contact details never reach the client; verified by automated tests.
- Delivered 26 automated tests covering validation, abuse controls, failure modes and provider request formats.
- Managed a 20+ page data-driven website with a single-file event data source that drives the homepage ticker, event listings and seasonal campaign pages.
- Coordinated multiple AI tools (design, coding, browser automation) with written specs and handoff documents to deliver production features without a development team.

9. Be ready to answer in interviews

- Why did you add a honeypot instead of a CAPTCHA? (Friction on mobile QR traffic; server still rate-limits.)
- What happens if WhatsApp is down? (Email still sends, RSVP accepted, error logged.)
- Why validate age on the server? (The browser can be bypassed.)
- How do you know a flyer worked? (Source parameter plus self-reported field.)
- What would you improve next? Honest answers: store submissions in a database for reporting; durable (shared-store) rate limiting instead of per-instance memory; Turnstile CAPTCHA if spam appears; confirm Resend domain authentication (SPF/DKIM); test against live Meta/Resend accounts; clean duplicate-case page files (`Contact.html` vs `contact.html`); add a privacy policy page.

10. Known gaps (do not claim these)

- Live Resend and WhatsApp delivery not yet tested end to end (only mocked).
- Shared persistent rate limiting is not implemented.
- No analytics dashboard or database yet.
- The `/kp` form does not collect age number, event choice, or FetLife username (your final design omitted them; the 19+ checkbox is the only age gate).
- The existing site has some duplicate files and a `mailto:` link on the Halloween page; the contact email in the repo is publicly visible by design.
- Some existing public pages carry explicit event names; keep that in mind for how you present the portfolio to different audiences.